Informe Técnico Integrador — Food Store

Materia: Base de Datos II
Trabajo: Trabajo Práctico Integrador (TPI) — Unidades 1 a 8
Integrantes: Julieta Herrera, María de los Ángeles Zalazar
Comisión: 1PROG2
Repositorio: https://github.com/juliiirnr-hash/TP2_BD2.git (incluye todas las unidades)

Resumen ejecutivo

El trabajo consistió en construir y evolucionar la base de datos **Food Store** a lo largo de ocho unidades, partiendo del modelado entidad-relación y la normalización hasta llegar a un sistema con optimización de consultas, indexado, vistas, roles, auditoría, respaldo/restauración, migración a MongoDB y una tablet SQLite con sincronización simulada.

El criterio metodológico común fue: **medir antes, proponer con IA, ejecutar, medir después y decidir con evidencia**. Ninguna propuesta generada por IA se incorporó sin haber sido ejecutada y contrastada contra el motor real (PostgreSQL, SQLite o MongoDB según la unidad). Los resultados principales fueron:

- Esquema relacional normalizado hasta BCNF, con restricciones de dominio, integridad referencial, borrado lógico e índices parciales.
- Reducciones de tiempo medidas en consultas optimizadas: hasta **−97 %** (14,7 ms → 0,38 ms) en el top-100 por stock y **6,05×** en la competencia de índices de la Unidad 2.
- Vistas y vista materializada para reportes, con control de acceso por roles y exclusión de datos sensibles.
- Roles y permisos de mínimo privilegio verificados con pruebas positivas y negativas; simulacro de incidente con escalamiento bloqueado.
- Respaldo lógico automatizado, restauración validada por conteos y reglas de negocio, con RPO del mismo día.
- Migración de una muestra de pedidos a MongoDB y comparación documentada PostgreSQL vs. MongoDB.
- Tablet SQLite con catálogo local, borradores, cola de sincronización y semáforo de estados, sin modificar el servidor.

---

1. Unidad 1 — Modelado, normalización, DDL y transacciones

1.1 Qué elementos se implementaron

- **Modelo entidad-relación (ER)** del dominio Food Store con cinco entidades: `USUARIO`, `CATEGORIA`, `PRODUCTO`, `PEDIDO` y `DETALLE_PEDIDO`.
- Resolución de la relación N:M entre pedidos y productos mediante la entidad asociativa `DETALLE_PEDIDO`, conservando `cantidad`, `precio_unitario` (snapshot del precio de compra) y `subtotal`.

- **Normalización hasta BCNF**, partiendo de una planilla desnormalizada y justificando dependencias funcionales para eliminar redundancias y anomalías de inserción, borrado y actualización.
- **Script DDL** `schema.sql` con:
- Tipos enumerados `rol_usuario`, `estado_pedido` y `forma_pago`.
- Claves primarias con `GENERATED ALWAYS AS IDENTITY`.
- Tipos precisos: `NUMERIC(10,2)` para dinero y `TIMESTAMPTZ` para fechas.
- Restricciones `CHECK` de dominio (`precio >= 0`, `stock >= 0`, `cantidad > 0`, `subtotal = cantidad * precio_unitario`).
- Restricciones `UNIQUE` (mail, nombre de categoría, combinación pedido-producto).
- Claves foráneas con `ON DELETE RESTRICT` y `ON DELETE CASCADE` según dependencia.
- Borrado lógico mediante `eliminado BOOLEAN`.
- Índices iniciales y parciales: `idx_pedido_usuario`, `idx_producto_categoria` con `WHERE eliminado = FALSE`, `idx_detalle_pedido`.
- **Trigger** `trg_pedido_estado_irreversible` y función `verificar_estado_pedido()` para impedir el retroceso de `CONFIRMADO` a `PENDIENTE`.
- **Experimentos de concurrencia y aislamiento** en DBeaver/PostgreSQL: lectura no repetible, espera por bloqueo y lectura fantasma.
- **Ejercicio de lectura crítica de SQL** sobre dos scripts generados: un `UPDATE` sin `WHERE` y un `DELETE ... NOT IN` con riesgo frente a `NULL`, reescrito con `NOT EXISTS`.

1.2 Cómo se probó

- Ejecución directa de `schema.sql` sobre PostgreSQL para verificar sintaxis, orden de dependencias y compatibilidad de tipos.
- Inserción de filas inválidas para comprobar el bloqueo por `CHECK` (precio negativo), por `UNIQUE` (mail duplicado) y por `FOREIGN KEY` (`ON DELETE RESTRICT`).
- Prueba del trigger con dos `UPDATE` dentro de una transacción: uno inválido (`CONFIRMADO` → `PENDIENTE`) y uno válido, con `ROLLBACK` final.
- Escenarios de concurrencia con dos pestañas de DBeaver:
- **Lectura no repetible:** `SELECT` en la pestaña 1, `UPDATE` + `COMMIT` en la pestaña 2, segundo `SELECT` en la pestaña 1.
- **Espera por bloqueo:** `SELECT ... FOR UPDATE` en la pestaña 1 sin cerrar transacción y el mismo `SELECT` en la pestaña 2.
- **Lectura fantasma:** `COUNT(*)` antes y después de un `INSERT` + `COMMIT` de otra sesión.
- Verificación de las explicaciones de la IA repitiendo cada experimento en el motor real.

1.3 Resultados obtenidos

- Esquema relacional consistente, sin redundancias y con restricciones que impiden datos inválidos.
- El trigger funciona: el retroceso de estado es rechazado con excepción y el avance de estado se acepta.
- Los escenarios de concurrencia mostraron el comportamiento esperado de PostgreSQL:
- Sin `REPEATABLE READ`, el segundo `SELECT` vio el valor actualizado (`800`); con `REPEATABLE READ`, mantuvo el valor original (`500,00`).

- `FOR UPDATE` dejó bloqueada la fila hasta el `COMMIT`; la segunda sesión quedó esperando y se destrabó al liberar el lock.
- La lectura fantasma se reprodujo: el `COUNT(*)` pasó de `1` a `2` tras el `INSERT` + `COMMIT` de otra sesión.
- Se detectó y documentó un error puntual de la IA al explicar la causa del fenómeno de lectura fantasma, que fue corregido con la verificación en el motor.

1.4 Consultas/índices optimizados y diferencias

- Se reemplazó la idea de un índice global por un **índice parcial** `idx_producto_categoria ON producto(id_categoria) WHERE eliminado = FALSE`.
- Diferencia observada: el índice parcial mantiene punteros solo a productos activos, reduciendo el tamaño en disco y memoria del B-Tree. Un índice global incluiría también los registros eliminados, sin beneficio para las consultas habituales del catálogo activo.
- En esta unidad no se registraron tiempos de `EXPLAIN ANALYZE`; la comparación fue estructural y se profundizó en las unidades 2 y 3.

---

2. Unidad 2 — Optimización de consultas (TP3 y TP4)

2.1 Qué elementos se implementaron

- **Carga masiva** de 50.000 productos con `generate_series`, precios entre 500 y 5000, stock entre 0 y 200 y distribución round-robin por categoría.
- `ANALYZE producto` posterior a la carga para actualizar estadísticas del optimizador.
- **Reescritura de tres consultas** sobre `producto`:
- **C1 (productos sobre el promedio):** subconsulta escalar reescrita con `AVG(precio) OVER ()`.
- **C2 (conteo por categoría):** incorporación de `WHERE eliminado = FALSE` para aprovechar índice parcial.
- **C3 (top-100 por stock):** incorporación de `WHERE eliminado = FALSE` e índice parcial `idx_producto_stock ON producto(stock DESC) WHERE eliminado = FALSE`.
- **Lectura crítica de planes**: análisis nodo por nodo de un plan con `WindowAgg` y contraste frase por frase con la explicación de la IA.
- **Consultas resumen y subconsultas bajo especificación**: agregación por categoría y productos con precio superior al promedio, con verificación de equivalencia mediante `EXCEPT` bidireccional.
- **Competencia de optimización**: cuatro propuestas de índices (A–D) para una consulta con `BETWEEN` y `ORDER BY`.

2.2 Cómo se probó

- Creación de una copia de trabajo `food_store_test` a partir de `food_store` y respaldo previo con `pg_dump`.
- Ejecución controlada con `BEGIN`, `\i script`, `COMMIT` / `ROLLBACK`, confirmando `INSERT 0 50000`.
- Medición con `EXPLAIN ANALYZE` antes y después de cada cambio, registrando plan, costo, tiempo real y buffers.
- Verificación de equivalencia de resultados con conteos, muestra de filas y `EXCEPT` en ambos sentidos.

- Contraste de la explicación de la IA contra el plan real, detectando supuestos no informados (por ejemplo, `work_mem` de 4 MB).

2.3 Resultados obtenidos

- Base masiva operativa de aproximadamente 50.000 productos, apta para medir `Seq Scan` reales.
- Tabla comparativa completa de planes antes/después.
- La reescritura C1 mejoró la estructura del plan (eliminó el doble escaneo del `InitPlan`) pero aumentó el tiempo por materialización de la ventana en disco: se documentó como no-mejora en tiempo.
- C2 y C3 mejoraron de forma clara.
- La competencia A–D quedó resuelta con datos: la propuesta C ganó por tiempo real, aunque se declaró su trade-off por ser hiperespecífica.

2.4 Consultas optimizadas y diferencias antes/después

Consulta	Antes	Cambio aplicado	Después	Diferencia
C1 — precio mayor al promedio	`Bitmap Heap Scan` + `InitPlan`, 22,9 ms	Reescritura con `AVG() OVER ()` + `ANALYZE`	`Subquery Scan` + `WindowAgg`, 113 ms	Plan más limpio y escalable, pero mayor tiempo por spill a disco
C2 — conteo por categoría	`Seq Scan` + `HashAggregate`, 22,5 ms	`WHERE eliminado = FALSE`	`Index Only Scan` + `GroupAggregate`, 12,4 ms	Buffers 820 → 42; −45 % de tiempo
C3 — top-100 por stock	`Seq Scan` + `Sort`, 14,7 ms	`WHERE eliminado = FALSE` + `idx_producto_stock`	`Index Scan` + `Limit`, 0,38 ms	Sin `Sort`; −97 % de tiempo; buffers 820 → 95
Competencia ej5	`Seq Scan` + `Sort` + `Filter`, ~51,4 ms	Propuesta C: índice parcial `precio DESC WHERE disponible AND id_categoria = 1`	`Index Only Scan`, 8,49 ms	6,05× más rápido (−83,5 %)

Propuestas descartadas en la competencia:

Propuesta	Índice	Resultado	Decisión
A	`(id_categoria, disponible, precio)`	Seguía `Seq Scan` + `Sort`, costo idéntico (~51 ms)	Rechazada: no cambiaba acceso ni costo

Propuesta	Índice	Resultado	Decisión
B	Índice covering con `INCLUDE (id_producto, nombre, stock)`	`Index Only Scan`, 9,26 ms	Rechazada: sobrecargado y costoso en escritura
C	`(precio DESC) WHERE disponible AND id_categoria = 1`	`Index Only Scan`, 8,49 ms	Aceptada por menor tiempo real
D	`(id_categoria, precio DESC) WHERE disponible`	`Index Scan`, 31,03 ms	Rechazada para la competencia; más versátil pero más lenta en este caso

> **Alcance:** la carpeta `u2tp4` contiene únicamente la consigna del TP4 (joins de 3+ tablas, `Nested Loop` / `Hash Join` / `Merge Join`, ranking con ventana y subconsulta correlacionada). No hay `queries.sql` ni mediciones ejecutadas, por lo que se documenta como pendiente y no se inventan resultados.

---

3. Unidad 3 — Indexado, vistas y vistas materializadas (TP5)

3.1 Qué elementos se implementaron

- **Tres índices B-Tree parciales** definidos a partir de especificaciones en Markdown:
- `idx_producto_precio_rango ON producto (id_categoria, precio) WHERE eliminado = FALSE`.
- `idx_categoria_producto_disponibilidad ON producto (id_categoria, disponible) WHERE eliminado = FALSE`.
- `idx_pedido_historial_articulos ON pedido (fecha DESC) WHERE eliminado = FALSE`.
- **Tres vistas para reportes**:
- `v_productos_vigentes`: producto + categoría, filtrando borrado lógico en ambas tablas.
- `v_pedidos_usuario`: pedido + usuario, excluyendo expresamente `contrasena`.
- `v_detalle_pedido_producto`: detalle + producto.
- **Control de acceso RBAC**: rol `rol_reportes` con `REVOKE SELECT` sobre las tablas base y `GRANT SELECT` solo sobre las vistas.
- **Vista materializada** `productos_caros_pcategoria`: top-10 de productos disponibles y vigentes con su categoría, `WITH DATA`, más índice único `pcpc_id` y política de `REFRESH`.

3.2 Cómo se probó

- `EXPLAIN ANALYZE` antes y después para las consultas 4, 5 y 6.
- Medición del impacto en escritura con un bloque anónimo `DO` que insertó 500 productos y 500 detalles (1.000 registros) antes y después de los índices.
- Comparación de cada vista contra su consulta manual equivalente, con `ORDER BY`, verificando coincidencia exacta de filas y valores.

- Verificación de seguridad: `rol_reportes` no puede leer tablas base y sí puede leer las vistas; `contrasena` no aparece en `v_pedidos_usuario`.
- Comparación de la consulta original contra la lectura de la vista materializada.

3.3 Resultados obtenidos

- Reportes abstraídos sin exponer tablas base ni columnas de auditoría.
- Acceso de reportes aislado con `rol_reportes`.
- Ranking top-10 servido desde la vista materializada sin recalcular `JOIN` + `ORDER BY` en cada lectura.
- Se documentó el costo de escritura de los índices y se descartó una propuesta de sobreindexación.

3.4 Consultas optimizadas y diferencias antes/después

Consulta	Antes (plan / tiempo / buffers)	Índice o cambio aplicado	Después	Diferencia
C4 — distribución de precios por categoría	`Seq Scan` + `Sort`, 112,706 ms, hit=820	`idx_producto_precio_rango`	`Index Only Scan`, 84,529 ms, hit=784 read=193	−25 % de tiempo; sin `Seq Scan`; rangos resueltos desde el B-Tree
C5 — disponibilidad por categoría	`Seq Scan` + `HashAggregate`, 43,014 ms, hit=820	`idx_categoria_producto_disponibilidad`	`Index Only Scan`, 52,122 ms, hit=15 read=43	Sin `Seq Scan`; buffers 820 → 15 hits; el tiempo puntual subió por lectura limpia
C6 — historial de pedidos con artículos	`Seq Scan` en pedido y detalle + `Sort`, 1,566 ms	`idx_pedido_historial_articulos`	`Index Scan` + `Index Only Scan` + `Incremental Sort`, 0,296 ms en caché / 2,065 ms en lectura limpia	Sin escaneos secuenciales ni `Sort` completo; orden provisto por `fecha DESC`
Top-10 de productos caros	Consulta original, ejecución 0,144 ms	Vista materializada + `pcpc_id`	Ejecución 0,110 ms	Lectura precomputada; exige `REFRESH` ante cambios

Impacto en escritura:

Escenario	Volumen	Tiempo	Variación
Antes (sin índices nuevos)	1.000 registros	63 ms	Base

Escenario	Volumen	Tiempo	Variación
Después (con índices de Parte A)	1.000 registros	138 ms	+119 % (+75 ms). Aceptable porque predominan las lecturas

Propuesta descartada por sobreindexación: `CREATE INDEX idx_producto_eliminado ON producto (eliminado)`. Se descartó porque:
1. Tiene baja selectividad: más del 99 % de las filas tiene `FALSE` y el optimizador prefiere `Seq Scan`.
2. Es redundante: los índices aceptados ya filtran con `WHERE eliminado = FALSE`.
3. Penaliza `INSERT` / `UPDATE` sin beneficio de lectura.

---

4. Unidad 4 — BCNF y desnormalización controlada

4.1 Qué elementos se implementaron

Parte 1 — Análisis y aplicación de BCNF:

- Esquema inicial `control_lote_almacen` con PK `(lote_id, deposito_id)` y reglas de negocio:
- `{LoteID, DepositoID} → ResponsableControlID`.
- `ResponsableControlID → DepositoID`.
- Cálculo de clausuras y claves candidatas; determinación de que el esquema **no cumple BCNF** porque `ResponsableControlID` es determinante pero no superclave.
- Análisis de las tres anomalías clásicas (inserción, borrado y actualización) con la instancia de ejemplo.
- **Descomposición sin pérdida** en:
- `responsable_deposito(responsable_control_id PK, deposito_id FK)`.
- `control_lote(lote_id, responsable_control_id, PK compuesta, FKs)`.
- **Vista de compatibilidad** `v_control_lote_almacen` que reconstruye la relación original mediante `NATURAL JOIN`.
- Migración de los datos de ejemplo: `responsable_deposito` con `(801, 30)` y `(802, 31)`; `control_lote` con `(501, 801)`, `(502, 801)` y `(503, 802)`.

Parte 2 — Desnormalización controlada:

- Consulta del panel: top-5 de categorías por monto vendido en el día, con `JOIN` de `detalle_pedido`, `producto`, `categoria` y `pedido`.
- **Vista materializada** `rancking_dia` con `SUM(dp.subtotal)`, filtros de fecha y borrado lógico, `ORDER BY total_vendido DESC LIMIT 5`.
- Índice único `ux_rancking_dia_categoria` para habilitar `REFRESH MATERIALIZED VIEW CONCURRENTLY`.
- Script de auditoría para detectar desincronización entre la vista y la fuente de verdad.

4.2 Cómo se probó

- Verificación de la reunión sin pérdida comparando la vista de compatibilidad contra la tabla original: conteo de 3 filas en ambas y dos `EXCEPT` (uno por sentido) con **0 filas**.
- `EXPLAIN ANALYZE` de la consulta original y de la consulta sobre la vista materializada.
- Ejecución del script de auditoría sobre la base migrada, esperando resultado vacío.

4.3 Resultados obtenidos

- La descomposición reconstruye exactamente la relación original y elimina la violación de BCNF.
- La vista materializada reduce fuertemente el costo de lectura del reporte.

4.4 Consulta optimizada y diferencias antes/después

Métrica	Antes (consulta sobre tablas base)	Después (vista materializada)
Tiempo de ejecución	13,548 ms	0,802 ms
Tiempo de planificación	10,084 ms	1,372 ms
Nodo dominante	`HashAggregate` (costo 801,36) + múltiples joins / `Seq Scan` filtrando 10.223 filas	`Seq Scan on rancking_dia` (costo 1,05)
Costo de I/O	`shared hit=1661`	`shared hit=1`

La auditoría de sincronización devolvió un conjunto vacío, confirmando que la vista coincidía con el cálculo en tiempo real.

> **Alcance:** en el repositorio se conserva el script de la Parte 1 (`descompocision_deposito.sql`) y el informe PDF con la Parte 2. La Parte 2 no está como script separado en la carpeta `u4` .

---

5. Unidad 5 — Roles, privilegios, auditoría y simulación de incidente

5.1 Qué elementos se implementaron

- **Cinco roles de grupo** `NOLOGIN` : `rol_app_lectura` , `rol_app_escritura` , `rol_soporte` , `rol_reportes` , `rol_catalogo_admin` .
- **Tres roles de login**: `app_web` , `backoffice_catalogo` , `admin_datos` , con atributos `LOGIN INHERIT NOSUPERUSER NOCREATEDB NOCREATEROLE` (salvo `admin_datos` , que tiene `CREATEDB` y `CREATEROLE`).
- **Membresías exactas**:
- `app_web` → `rol_app_lectura` + `rol_app_escritura` .
- `backoffice_catalogo` → `rol_app_lectura` + `rol_catalogo_admin` .
- `admin_datos` no es miembro de ningún grupo: es owner.
- **Objetos nuevos**:

- `v_categorias_vigentes`.
- `v_usuarios_publico` (sin `contrasena`).
- `v_pedidos_resumen` (sin `contrasena`).
- `mv_ventas_categoria_mes` (materializada mensual por categoría) con índice único para `REFRESH CONCURRENTLY`.
- `sp_crear_pedido(...)` en `SECURITY INVOKER` para alta atómica de pedido con validación de productos vigentes y cálculo de subtotales/total.
- **Permisos de mínimo privilegio**:
- `REVOKE ALL` previo a cada `GRANT`.
- `GRANT` por columna en `pedido` (`estado`), `detalle_pedido` (`cantidad`, `eliminado`), `producto` y `categoria`.
- Sin `DELETE` ni `TRUNCATE` para roles de aplicación.
- Secuencias restringidas solo a las necesarias.
- **Auditoría básica** con `log_connections = on`, `log_disconnections = on` y `log_statement = 'mod'`.
- **Simulacro de incidente**: usuario sintético, tres intentos fallidos de login, un login exitoso, lectura masiva como `app_web`, intento de `GRANT` a `admin_datos` y `TRUNCATE` de `usuario`.
- **Anonimización**: consulta de usuarios por rol y mes de alta sobre `usuario_anon` y sobre `usuario`, con resultados equivalentes.
- **Bitácora de IA** con prompts, respuestas, validaciones y decisiones.

5.2 Cómo se probó

- Re-ejecución completa de `roles.sql` dos veces para verificar idempotencia.
- Consultas a `pg_roles` y `pg_auth_members` para comprobar `rolcanlogin`, `rolsuper` y membresías.
- Pruebas positivas y negativas con `SET ROLE`:
- `app_web` puede leer las vistas de catálogo; no puede leer tablas base ni insertar productos.
- `backoffice_catalogo` puede actualizar catálogo; no puede insertar pedidos.
- `rol_soporte` lee `v_usuarios_publico` sin la columna `contrasena`.
- `rol_reportes` lee las vistas y la materializada.
- Verificación de la configuración de auditoría con `SHOW log_connections`, `SHOW log_disconnections` y `SHOW log_statement`.
- Ejecución del simulacro sentencia por sentencia en DBeaver, ya que los meta-comandos `\set` / `\echo` de `psql` no funcionan en ese cliente.
- Contraste de la respuesta de la IA sobre el log anonimizado contra lo realmente ejecutado.

5.3 Resultados obtenidos

- Esquema de roles y permisos de mínimo privilegio, re-ejecutable y verificado.
- El escalamiento **falla**:
- `GRANT admin_datos TO app_web` → permiso denegado.
- `TRUNCATE TABLE usuario` como `app_web` → permiso denegado.
- Lectura masiva de `usuario` como `app_web` → permiso denegado.

- La auditoría registra conexiones, desconexiones y sentencias de modificación; no registra `SELECT` por la elección de `log_statement = 'mod'`.
- La consulta de anonimización devolvió los mismos agregados en `usuario_anon` y `usuario` (ADMIN: 1; USUARIO: 4; mes 2026-09).
- Se documentó que los logs son un activo sensible y que la auditoría nativa no cubre detalle fila por fila, usuario final real ni lecturas; `pgaudit` cubriría parte de esa brecha.

5.4 Consultas/índices optimizados

En esta unidad no se realizó una optimización de consultas con `EXPLAIN ANALYZE`; el foco fue seguridad, control de acceso y auditoría. La única consulta relevante fue la de anonimización, que se validó por equivalencia de resultados, no por rendimiento.

---

6. Unidad 6 — Mantenimiento, respaldo y recuperación

6.1 Qué elementos se implementaron

- **Script de respaldo** `backup_foodstore.ps1`:
- `pg_dump` en formato custom (`-Fc`) con compresión (`-Z 6`).
- Nombre dinámico con fecha y hora: `food_store_test_YYYYMMDD_HHmm.dump`.
- Verificación de legibilidad con `pg_restore --list`.
- Renombrado atómico desde un archivo `.parcial`.
- Huella SHA-256 guardada en `.sha256`.
- Retención configurable (14 días).
- Logs dedicados y salida con código de error si falla.
- Conexión con rol `mant_backup` y sin contraseñas en texto plano (uso de `.pgpass` /variables de entorno).
- **`health_check.sql`**: tamaño de la base, conexiones y transacciones, estado de tablas y tuplas muertas, consultas activas de más de 5 minutos, tamaño por tabla e índices.
- **`maintenance.sql`**: revisión de `last_autovacuum` / `last_autoanalyze` y `ANALYZE` justificado sobre `detalle_pedido`, `pedido` y `producto`; se evita `VACUUM FULL`.
- **Simulacro de restauración** sobre `foodstore_restore_test` con `pg_restore`, validación de conteos y reglas lógicas de negocio.

6.2 Cómo se probó

- Ejecución manual del script de respaldo en PowerShell, verificando la creación del `.dump`, el `.sha256` y el log.
- Restauración con `pg_restore --no-owner --no-privileges` sobre una base descartable.
- Conteo de filas por tabla y ejecución de consultas de integridad:
- `productos_invalidos = 0` (`precio < 0 OR stock < 0`).
- `detalles_invalidos = 0` (`cantidad <= 0 OR precio_unitario < 0`).
- Revisión del bloque `catch` del script, donde se detectó y corrigió un índice de formateo inconsistente.

6.3 Resultados obtenidos

Conteos recuperados:

Tabla	Filas
`categoria`	9
`usuario`	20
`producto`	50
`pedido`	10.700
`detalle_pedido`	53.500

- Restauración finalizada sin errores y con conteos consistentes.
- `productos_invalidos = 0` y `detalles_invalidos = 0`.
- **RPO observado:** cercano a 0 horas, porque se usó el respaldo más reciente del mismo día (29/10/2026, 09:52).
- **RTO:** el objetivo de laboratorio era menos de 1 hora; la restauración terminó sin errores, pero el tiempo numérico no quedó registrado en la evidencia de `u6`. Se dejó explícitamente como "no registrado" en lugar de inventarlo.
- Mejora propuesta: agregar medición automática con `Measure-Command` y validación post-restore que registre RTO y RPO en el log.

6.4 Consultas optimizadas

No se optimizaron consultas en esta unidad. Las consultas de `health_check.sql` y `maintenance.sql` son de diagnóstico y mantenimiento, no de rendimiento de negocio.

---

7. Unidad 7 — Migración y comparación PostgreSQL vs. MongoDB

7.1 Qué elementos se implementaron

- Migración de una muestra de **5 pedidos** (IDs 10108 a 10112) desde PostgreSQL a MongoDB.
- Modelo documental con un documento por pedido y el detalle embebido en un arreglo `detalle_pedido`, conservando `_id` igual al `id_pedido` original.
- Archivos:
- `esquema_parte1.json`: estructura del documento.
- `migracion_y_consultas.js`: inserción de documentos y consultas en `mongosh`.
- `analisis_pg_vs_mongo.md`: análisis arquitectónico completo.
- Consultas equivalentes en ambos motores:
- Filtro por estado `PENDIENTE`.
- Búsqueda por `id_producto = 10`.

- Filtro por `total > 20000`.
- Análisis de ventajas y desventajas de cada motor y recomendación técnica.

7.2 Cómo se probó

- Ejecución de las consultas en PostgreSQL (DBeaver) y en MongoDB (`mongosh`/Compass).
- Comparación de cantidades, IDs devueltos y estructura del resultado.
- Contraste de las afirmaciones de la IA contra la documentación oficial de MongoDB y contra el laboratorio.

7.3 Resultados obtenidos

Consulta / filtro	PostgreSQL	MongoDB	Coincidencia de IDs
Estado `PENDIENTE`	15 filas	3 documentos	Sí: 10110, 10111, 10112
Producto `id_producto = 10`	1 fila	1 documento	Sí: 10112
`total > 20000`	5 filas	1 documento	Sí: 10112

La diferencia de "15 filas" frente a "3 documentos" se explica porque PostgreSQL devuelve el resultado plano del `JOIN` con el detalle, duplicando la cabecera por cada renglón, mientras que MongoDB devuelve un documento unificado por pedido. La afirmación de la IA sobre esta duplicación se verificó en el laboratorio: 25 filas planas para 5 pedidos en PostgreSQL frente a 5 documentos en MongoDB.

Conclusiones del análisis:

- MongoDB es más eficiente para leer un pedido completo como documento único, sin `JOIN`.
- PostgreSQL conserva mejor la coherencia financiera y referencial mediante `CHECK`, `NUMERIC(10,2)`, FKs y transacciones ACID.
- La recomendación del informe de la Unidad 7 fue una arquitectura híbrida: PostgreSQL como base OLTP primaria y MongoDB como almacén de lectura del historial de pedidos confirmados.
- En la Unidad 8 se retomó la conclusión y se aclaró que MongoDB quedó como copia de estudio, fuera del circuito de venta: la fuente de verdad es PostgreSQL.

7.4 Consultas optimizadas

No se aplicaron índices ni reescrituras de optimización en esta unidad. El trabajo fue comparativo: se midió la diferencia estructural entre un resultado relacional plano y un documento embebido, no una mejora de tiempo dentro del mismo motor.

---

8. Unidad 8 — Food Store en la feria: tablet SQLite

8.1 Qué elementos se implementaron

- **Esquema SQLite** `schema_sqlite.sql` con cuatro tablas `STRICT`:
- `producto_cache`: copia de solo lectura del catálogo.

- `pedido_borrador`: cabecera del borrador local.
- `item_borrador`: líneas del borrador, con PK compuesta y FKs.
- `cola_sync`: solicitudes de sincronización con estados `PENDIENTE`, `CONFLICTO`, `RECHAZADO`, `SIMULADO_APROBADO`.
- Restricciones `CHECK` para precios, stock, cantidades, disponibilidad, borrado lógico y forma de pago.
- `PRAGMA foreign_keys = ON` por conexión y `DROP TABLE IF EXISTS` para re-ejecución.
- **Vista PostgreSQL** `v_snapshot_movil` y exportación a CSV de la instantánea del servidor.
- **`pruebas_sqlite.sql`**: borrador completo con `COMMIT`, operación deshecha con `ROLLBACK`, caso C1 (catálogo disponible sin servidor) y casos C7 (cantidad 0 y producto inexistente, con `SAVEPOINT` / `ROLLBACK TO`).
- **`semaforo.sql`**: clasificación de solicitudes `PENDIENTE` contra `snapshot_servidor` en rojo, amarillo o verde, sin escribir en PostgreSQL.
- **Code review con IA** de scripts de la cursada y correcciones documentadas.

8.2 Cómo se probó

- Verificación del esquema con `PRAGMA table_list` (las cuatro tablas con `strict = 1`) y `PRAGMA foreign_key_list(item_borrador)` (dos FKs).
- Ejecución real de `pruebas_sqlite.sql` sobre una copia de la base, comprobando los errores esperados de C7 y que el conteo de filas no cambiara.
- Carga del CSV en una tabla de paso `snapshot_servidor` y migración a `producto_cache` con conversión de tipos.
- Ejecución del semáforo con siete solicitudes que cubren todos los estados; repetición tres veces para verificar idempotencia.
- Verificación del servidor antes y después con `verificar_servidor.sql`: pedidos, detalles y stock total debían ser idénticos.
- Comprobación de hallazgos del code review con consultas reales:
- `SELECT COUNT(*) FROM producto_cache` antes y después de las pruebas.
- `Parse error` al ejecutar el semáforo sin `snapshot_servidor`.
- `EXPLAIN` para comprobar que `DESC` en un índice es redundante.

8.3 Resultados obtenidos

Estados del semáforo:

Caso	Preparación	Resultado
C2	Borrador del producto 2 a 265787 centavos; en el servidor se subió el precio a 266287	`CONFLICTO — cambio el precio: 2`
C2 (acepta)	Se actualizó el precio del borrador y se creó la solicitud `S-C2b`	`SIMULADO_APROBADO — todo coincide`

Caso	Preparación	Resultado
C3	Borrador de 10 unidades del producto 4 (stock visto: 3)	`RECHAZADO — stock insuficiente: 4`
C4	Baja lógica del producto 5 en el servidor	`RECHAZADO — dado de baja: 5`
C4 (recarga)	Se recargó `producto_cache` desde la instantánea nueva	La fila queda con `eliminado = 1`, pero no aparece en el catálogo disponible
C5	Borrador del producto 1 sin cambios	`SIMULADO_APROBADO — todo coincide`
C6	Se reenvió la solicitud `S-C5` (mismo UUID)	`UNIQUE constraint failed: cola_sync.solicitud_uuid`

- El servidor no cambió: pedidos 7 → 7, detalles 10 → 10, stock total 5.007.341 → 5.007.341.
- `SIMULADO_APROBADO` no es una venta: no se escribió nada en PostgreSQL ni se descontó stock.
- Hallazgos del code review:
- `pruebas_sqlite.sql` borra `producto_cache`; se documentó y se corre sobre una copia. El conteo pasó de 50.001 a 6 al ejecutarlo sobre la base entregada.
- `semaforo.sql` usaba `snapshot_servidor`, que no crea `schema_sqlite.sql`; se agregó el paso de importación al README.
- El `DESC` del índice `idx_producto_stock` es redundante: PostgreSQL puede recorrer el B-Tree al revés (`Index Scan Backward`). Se descartó el cambio.
- `usuario.contrasena` está en texto plano: se aceptó como deuda técnica real, fuera del alcance del TP8.

8.4 Consultas optimizadas

- No se optimizaron consultas de negocio en SQLite. El foco fue integridad local, idempotencia y simulación.
- En el code review se evaluó el índice `idx_producto_stock`: se comprobó con `EXPLAIN` que un índice ascendente produce el mismo plan (`Index Scan Backward`) que uno con `DESC`, por lo que el cambio se descartó.

---

9. Cuadro consolidado de optimizaciones

Unidad	Consulta / estructura	Antes	Después	Diferencia
U1	Índice de catálogo por categoría	Índice global (incluye eliminados)	Índice parcial `WHERE eliminado = FALSE`	Menor tamaño en disco/memoria; lecturas más eficientes del catálogo activo

Unidad	Consulta / estructura	Antes	Después	Diferencia
U2	C1 — precio mayor al promedio	22,9 ms	113 ms	Plan más limpio y escalable, pero mayor tiempo por spill de `WindowAgg`
U2	C2 — conteo por categoría	22,5 ms	12,4 ms	−45 %; buffers 820 → 42
U2	C3 — top-100 por stock	14,7 ms	0,38 ms	−97 %; buffers 820 → 95; sin `Sort`
U2	Competencia ej5	~51,4 ms	8,49 ms	6,05× más rápido (−83,5 %)
U3	C4 — distribución de precios	112,706 ms	84,529 ms	−25 %; sin `Seq Scan`
U3	C5 — disponibilidad por categoría	43,014 ms	52,122 ms	Sin `Seq Scan`; buffers 820 → 15 hits; tiempo puntual mayor por lectura limpia
U3	C6 — historial de pedidos	1,566 ms	0,296 ms en caché / 2,065 ms limpia	Sin escaneos secuenciales ni `Sort` completo
U3	Top-10 productos caros	0,144 ms	0,110 ms	Lectura precomputada desde materializada
U4	Top-5 categorías del día	13,548 ms	0,802 ms	−94 %; buffers 1661 → 1

Lectura de conjunto: los mayores beneficios aparecieron con índices parciales alineados a los filtros reales (`eliminado = FALSE`, `disponible = TRUE`, `id_categoria`) y con la precomputación de agregaciones en vistas materializadas. También se documentaron casos donde la reescritura mejora la estructura pero no el tiempo puntual (C1) y donde el tiempo sube por una lectura limpia de disco (C5), en lugar de presentar solo los casos favorables.

---

10. Uso de herramientas de IA

10.1 OpenCode (herramienta indicada por la cátedra)

Se usó como agente principal para generar, revisar y corregir scripts SQL, esquemas, pruebas y documentación. El criterio fue siempre **evidencia antes que síntesis**: ejecutar en el motor y comparar

resultados antes de aceptar una respuesta.

Decisiones aceptadas:

- Tipos enumerados nativos, `CHECK` de subtotal, trigger de estado irreversible y borrado lógico en el esquema inicial.
- Índices parciales de las Unidades 2 y 3, vistas y vista materializada de la Unidad 3.
- Roles, permisos por columna y `ALTER DEFAULT PRIVILEGES` de la Unidad 5.
- Script de respaldo, `health_check.sql` y `maintenance.sql` de la Unidad 6.
- Esquema SQLite, pruebas, semáforo y correcciones del code review de la Unidad 8.

Decisiones descartadas, corregidas o limitadas:

Unidad	Propuesta o respuesta de la IA	Decisión del equipo	Motivo
U1	Explicación de la causa de la lectura fantasma	Corregida	La verificación en el motor mostró un error puntual en la causa atribuida por la IA
U2	Script de 50.000 productos con `&&` en PowerShell	Corregido	`&&` no es separador válido en PowerShell; se usó `OFFSET % COUNT`
U2	Reescritura C1	Aceptada con reserva	Mejoró la estructura pero empeoró el tiempo por spill a disco; se documentó la no-mejora
U2	Competencia A, B y D	Rechazadas	A no cambiaba el plan; B era sobrecargada; D era más lenta en este caso
U2	Explicación del plan `WindowAgg`	Aceptada con corrección	Asumió `work_mem = 4 MB` sin que se le hubiera informado
U3	`idx_producto_eliminado`	Descartada	Baja selectividad, redundancia y penalización de escritura
U5	`GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public`	Rechazada y restringida	Violaba mínimo privilegio; se limitó a las secuencias necesarias
U5	Script con `\set` / `\echo` de `psql`	Corregido	DBeaver devolvía error cerca de `\`; se ejecutó sentencia por sentencia
U5	Suposición sobre la estructura de `usuario_anon`	Corregida	La IA asumió columnas sin evidencia; se consultó `information_schema.columns` y se rehízo la consulta

Unidad	Propuesta o respuesta de la IA	Decisión del equipo	Motivo
U6	Script de respaldo y PDF del informe	Aceptado con revisión	Se contrastaron fechas, comandos, conteos y resultados contra los archivos fuente; los tiempos no registrados se dejaron como "no registrados"
U8	`pruebas_sqlite.sql` que borra `producto_cache`	Corregido y documentado	Se advirtió y se ejecuta sobre una copia
U8	`semaforo.sql` sin crear `snapshot_servidor`	Corregido	Se agregó el paso de importación al README
U8	Quitar `DESC` del índice `idx_producto_stock`	Descartado	`EXPLAIN` mostró el mismo plan (`Index Scan Backward`)
U8	`usuario.contrasena` en texto plano	Aceptado como deuda	Es un problema real, pero fuera del alcance del TP8

10.2 Kiro

- Se usó para **especificaciones y steering** del proyecto, especialmente en la Unidad 5 (`requirements.md` , `design.md`) y en la organización de la Unidad 8 (consigna 4.a).
- Finalidad: estructurar requisitos, roles, objetos a crear, criterios de aceptación y decisiones de diseño antes de generar el SQL.
- Decisión: se aprovecharon las especificaciones como guía; la consigna 4.a de la Unidad 8 no se entregó. El código final se validó igualmente contra la base, no contra el texto de la especificación.

10.3 Devin y Claude

- **Devin** se utilizó en la **competencia de optimización de la Unidad 2** como generador complementario de propuestas de índices (A–D), ampliando el espacio de alternativas para la consulta con `BETWEEN` y `ORDER BY` .
- **Claude** se usó para **consultas puntuales** de verificación y aclaraciones técnicas durante el desarrollo, siempre contrastadas contra el motor real o la documentación oficial.
- Decisiones:
- **Aceptada la propuesta C** por ser la de menor tiempo real medido (8,49 ms).
- **Descartadas A, B y D** por no mejorar el plan, por sobrecarga de escritura o por ser más lentas en este caso.
- Todas las propuestas se ejecutaron y midieron con `EXPLAIN ANALYZE` ; ninguna se aceptó por la reputación o la explicación de la herramienta.

10.4 Criterio general de aceptación

1. La IA propone; el equipo ejecuta.
2. Toda afirmación técnica se contrasta contra el motor real o contra la documentación oficial.
3. Si una propuesta mejora la estructura pero empeora el tiempo, se documenta el trade-off en lugar de ocultarlo.
4. Si no hay evidencia de un dato (por ejemplo, RTO numérico o una columna inexistente), se deja como pendiente o "no registrado"; no se inventa.
5. No se enviaron datos personales reales a las herramientas de IA. Los datos usados fueron ficticios y, cuando correspondió, anonimizados (`usuario_anon`, mails `test.local`, IPs `192.0.2.x`).

---

11. Conclusiones

- El proyecto evolucionó de un esquema relacional normalizado a un sistema con optimización medida, control de acceso, auditoría, respaldo y una extensión móvil, manteniendo PostgreSQL como fuente de verdad.
- Las mejoras de rendimiento más relevantes se obtuvieron con índices parciales y vistas materializadas, no con índices genéricos: el caso del top-100 por stock (−97 %) y el ranking diario (−94 %) son los ejemplos más claros.
- La verificación en el motor fue determinante para detectar errores de la IA (causa de la lectura fantasma, estructura supuesta de `usuario_anon`, comandos de `psql` en DBeaver, `DESC` redundante) y para evitar incorporar propuestas sin respaldo.
- La seguridad se abordó en capas: mínimo privilegio, vistas que ocultan datos sensibles, auditoría y simulación de incidente. El escalamiento y las lecturas masivas no autorizadas quedaron bloqueados.
- La restauración se validó con conteos y reglas de negocio; la deuda principal es la falta de registro numérico del RTO y la automatización completa de esa medición.
- La tablet SQLite cumplió el principio "el móvil propone, el servidor dispone": ningún estado `SIMULADO_APROBADO` modificó precio, stock o ventas.

---

12. Anexo — Evidencia y alcance

Unidad	Evidencia principal
U1	`u1/u1tp2/schema.sql`, `informe_ocurrencia.md`, `DUIA.md`, `ejercicio_lectura_critica.md`, `u1/u1tp1/schema.sql`
U2	`u2/u2tp3/insertar_productos.sql`, `consultas_ej2_TP3.sql`, `ej1TP3_cargadatos.md`, `capturas_TP3_ej1/`, PDFs de cuadros y competencia
U3	`u3/u3tp5/indices.sql`, `vistasMaterializadas.sql`, `informe_mediciones.md`, `informe_resolucion4.2B.md`, `specs/`

Unidad	Evidencia principal
U4	`u4/tp1u4GRUPOB.rar` → `tp1/descompocision_deposito.sql` y `TP_U4_GrupoB.pdf`
U5	`u5/sql/roles.sql`, `vistas.sql`, `simulacro_incidente.sql`, `u5/docs/`, `u5/bitacora.md`, `u5/parteD_TPu5.sql`
U6	`u6/backup_foodstore.ps1`, `health_check.sql`, `maintenance.sql`, `informe_u6.pdf`, `Parte 2/`, `Parte 4/`
U7	`u7/esquema_parte1.json`, `migracion_y_consultas.js`, `analisis_pg_vs_mongo.md`, informe PDF
U8	`u8/movil/schema_sqlite.sql`, `pruebas_sqlite.sql`, `semaforo.sql`, `u8/docs/`, `u8/README.md`

Alcance no ejecutado o no conservado en el repositorio:

- TP4 de la Unidad 2: solo consigna, sin `queries.sql` ni mediciones.
- Parte 2 de la Unidad 4: documentada en el PDF; no hay script separado en la carpeta.
- RTO numérico de la Unidad 6: no registrado en la evidencia.
- Consigna 4.a de la Unidad 8 (Kiro): no entregada.